



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ
по дисциплине «Разработка баз данных»

Практическое занятие №7

Студенты группы *ИКБО-20-23 Комисарик М.А.*

(подпись)

Ассистент *Брайловский А.В.*

(подпись)

Отчет представлен «__»_____2025 г.

Москва 2025 г.

ПОСТАНОВКА ЗАДАЧИ

Цель работы: Формирование углубленных практических навыков по управлению данными и реализации сложной бизнес-логики в СУБД PostgreSQL с использованием триггеров и курсоров.

Постановка задачи:

1. Анализ и оптимизация.

Определить три различных «медленных» запроса к вашей БД, которые можно оптимизировать с помощью разных типов индексов.

Для каждого из 3-х сценариев:

- 1) Выполнить анализ запроса «КАК ЕСТЬ» с помощью EXPLAIN ANALYZE.
- 2) Привести план выполнения «ДО», письменно проанализировать его и выявить причину низкой производительности.
- 3) Создать необходимый INDEX для оптимизации.
- 4) Повторно выполнить EXPLAIN ANALYZE.
- 5) Привести план выполнения «ПОСЛЕ», демонстрирующий использование индекса.
- 6) Сформировать сравнительную таблицу, демонстрирующую разницу в производительности «ДО» и «ПОСЛЕ».

2. Демонстрация атомарной транзакции (COMMIT). Реализуйте в своей базе одну бизнес-операцию внутри транзакции BEGIN...COMMIT.

3. Демонстрация отката транзакции (ROLLBACK). Адаптировать SQL-скрипт, моделирующий сбой операции, под свою предметную область, повторив описанные действия.

4. Моделирование аномалии «Неповторяемое чтение». Используя два редактора SQL, смоделировать проблему «неповторяемое чтение» на уровне изоляции по умолчанию (READ COMMITTED).

5. **Устранение аномалии «Неповторяемое чтение».** Повторить моделирование из Задания №4, но с использованием уровня изоляции REPEATABLE READ.

ХОД РАБОТЫ

Начальные данные

	123 id_employee	123 id_job_position	A-Z first_name	A-Z second_name	A-Z third_name	A-Z app_account_password_hash	A-Z phone_number	
1	1	1	Alice	Johnson	[NULL]	emp_hash_1	555-3333	1
2	2	2	Bob	Williams	Lee	emp_hash_2	555-4444	2
3	3	3	Carol	Martinez	[NULL]	emp_hash_3	555-8888	3
4	4	4	Dave	Anderson	Paul	emp_hash_4	555-9999	4
5	5	5	Eva	Garcia	Maria	emp_hash_5	555-0000	5
6	6	2	Frank	Taylor	[NULL]	emp_hash_6	555-1212	6
7	7	3	Grace	Thomas	Elizabeth	emp_hash_7	555-1313	7

Рисунок 1 – Содержание таблицы employee

	123 id_job_position	A-Z name	A-Z salary	
1	1	Manager	50000	
2	2	Chef	45000	
3	3	Delivery Driver	30000	
4	4	Cashier	28000	
5	5	Assistant Manager	42000	

Рисунок 2 – Содержание таблицы job_position

	123 id_product	A-Z name	A-Z description	123 price	
1	1	Margherita Pizza	Classic pizza with tomato and cheese	\$10.99	
2	2	Pepperoni Pizza	Pizza with pepperoni slices	\$12.99	
3	3	Cola	Refreshing soft drink	\$2.99	
4	4	Veggie Supreme	Pizza with assorted vegetables	\$14.99	
5	5	Chocolate Cake	Rich chocolate dessert	\$6.99	
6	6	Caesar Salad	Fresh salad with Caesar dressing	\$8.99	
7	7	Garlic Bread	Toasted bread with garlic butter	\$4.99	
8	8	Sprite	Sprite	\$3.49	
9	9	Lipton tea	A 1 liter bottle of Lipton tea	\$4.49	
10	10	Chocolate muffin	Tasty little chocolate dessert	\$4.99	
11	11	Chocolate donut	A chocolate glazed donut	\$5.49	

Рисунок 3 – Содержание таблицы product

	123 id_category	A-Z name	A-Z description	
1	1	Pizza	Various types of pizzas	
2	2	Beverage	Drinks and beverages	
3	3	Appetizer	Starters and snacks	
4	4	Dessert	Sweet treats	
5	5	Salad	Fresh salads	

Рисунок 4 – Содержание таблицы category

	123 id_product_ingredient	123 id_product	123 id_ingredient	123 ingredient_weight
1	1	1	1	500
2	2	1	2	200
3	3	1	3	150
4	4	2	1	500
5	5	2	2	200
6	6	2	3	150
7	7	2	4	100
8	8	4	1	500
9	9	4	2	200
10	10	4	3	150
11	11	4	5	80
12	12	4	6	70
13	13	4	7	60

Рисунок 5 – Содержание таблицы product_ingredient

	123 id_product_category	123 id_product	123 id_category
1	1	1	1
2	2	2	1
3	3	3	2
4	4	4	1
5	5	5	4
6	6	6	5
7	7	7	3
8	8	8	2
9	9	9	2
10	10	10	4
11	11	11	4

Рисунок 6 – Содержание таблицы product_category

	123 id_ingredient	A-Z name
1	1	Flour
2	2	Cheese
3	3	Tomato Sauce
4	4	Pepperoni
5	5	Mushrooms
6	6	Onions
7	7	Olives

Рисунок 7 – Содержание таблицы ingredient

	123 id_pizzeria	A-Z phone_number	123 id_address
1	1	555-0001	1
2	2	555-0002	2
3	3	555-0003	3
4	4	555-0004	4
5	5	555-0005	5

Рисунок 8 – Содержание таблицы pizzeria

	123 id_pizzeria_storage	123 id_pizzeria	123 id_storage	
1	1	1	1	
2	2	2	2	
3	3	3	3	
4	4	4	4	
5	5	5	5	

Рисунок 9 – Содержание таблицы pizzeria_storage

	123 id_storage_ingredient	123 id_storage	123 id_ingredient	123 ingredient_weight
1	4	2	4	2 000
2	5	3	5	1 500
3	6	4	6	1 800
4	7	5	7	1 200
5	2	1	2	5 000
6	1	1	1	10 000
7	3	1	3	3 000

Рисунок 10 – Содержание таблицы storage_ingredient

	123 id_client	AZ first_name	AZ second_name	AZ third_name	AZ phone_number	123 id_delivery_address	AZ account_password_id_hash
1	1	John	Doe	[NULL]	555-1111	1	hashed_password_123
2	2	Jane	Smith	Marie	555-2222	2	hashed_password_456
3	3	Mike	Johnson	Robert	555-3333	3	hashed_password_789
4	4	Sarah	Wilson	[NULL]	555-4444	4	hashed_password_101
5	5	David	Brown	James	555-5555	5	hashed_password_112
6	6	Emily	Davis	Anne	555-6666	6	hashed_password_131
7	7	Chris	Miller	Thomas	555-7777	7	hashed_password_415

Рисунок 11 – Содержание таблицы client

	123 id_address	AZ city	AZ street	123 house_number	123 floor	123 unit_number
1	1	New York	Main St	123	[NULL]	[NULL]
2	2	Los Angeles	Oak Ave	456	3	5
3	3	Chicago	Pine Rd	789	2	[NULL]
4	4	Boston	Maple St	321	1	2
5	5	Seattle	Cedar Ave	654	4	8
6	6	Miami	Palm Blvd	987	[NULL]	[NULL]
7	7	Denver	Elm St	147	2	3

Рисунок 12 – Содержание таблицы address

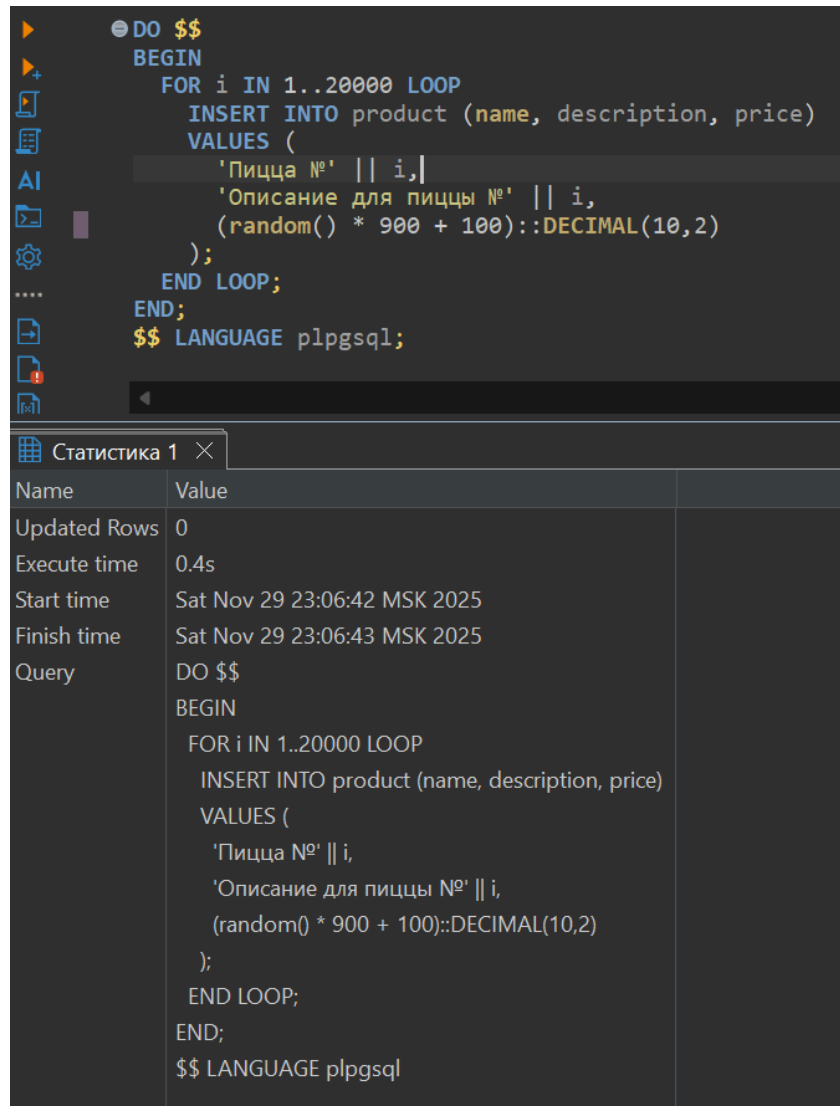
	123 id_order	123 id_pizzeria	123 id_client	123 id_delivery_address	123 order_amount	date_of_formation	123 delivery_method_id	123
1	4	1	4	4	\$18.98	2025-09-08	1	1
2	8	1	1	1	\$38.97	2025-11-21	2	1
3	11	1	4	4	\$45.97	2025-06-16	1	2
4	12	1	5	5	\$32.97	2025-06-10	2	1
5	1	1	1	1	\$25.97	2025-11-05	2	3
6	2	1	2	2	\$15.98	2025-02-06	1	2
7	3	1	3	3	\$32.97	2025-09-07	2	3
8	5	1	5	5	\$45.96	2025-02-09	2	2
9	6	1	6	6	\$12.99	2025-06-10	1	3
10	7	1	7	7	\$28.97	2025-09-11	1	4
11	9	1	2	2	\$22.98	2025-02-07	1	2
12	10	1	3	3	\$19.98	2025-11-21	2	1
13	15	1	1	1	\$32.97	2025-11-03	1	3

Рисунок 13 – Содержание таблицы _order.

	123 id_order_product	123 id_order	123 id_product	123 product_count
1	1	1	1	2
2	2	1	3	1
3	3	2	2	1
4	4	3	4	2
5	5	3	3	2
6	6	4	6	1
7	7	4	7	2
8	8	5	5	3
9	9	5	3	3
10	10	6	2	1
11	11	7	1	1
12	12	7	4	1
13	13	7	3	2
14	14	8	2	2
15	15	8	3	2
16	16	9	4	1
17	17	9	7	3
18	18	10	6	2
19	19	10	8	1
20	20	11	1	3
21	21	11	3	1
22	22	12	5	2
23	23	12	9	2
24	24	12	11	1

Рисунок 14 – Содержание таблицы order_product.

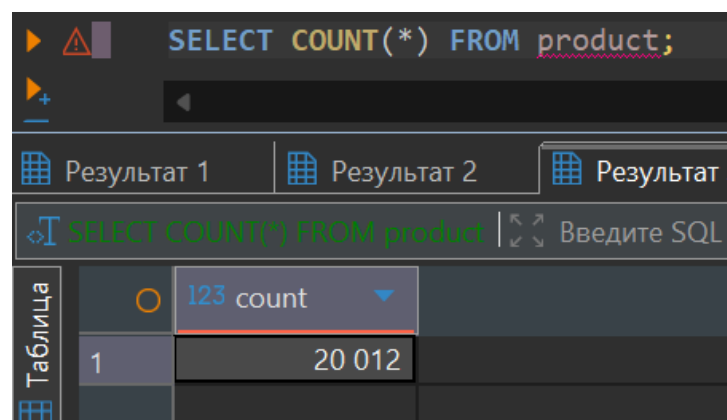
Подготовка базы данных



```
DO $$
BEGIN
  FOR i IN 1..20000 LOOP
    INSERT INTO product (name, description, price)
    VALUES (
      'Пицца №' || i,
      'Описание для пиццы №' || i,
      (random() * 900 + 100)::DECIMAL(10,2)
    );
  END LOOP;
END;
$$ LANGUAGE plpgsql;
```

Name	Value
Updated Rows	0
Execute time	0.4s
Start time	Sat Nov 29 23:06:42 MSK 2025
Finish time	Sat Nov 29 23:06:43 MSK 2025
Query	DO \$\$ BEGIN FOR i IN 1..20000 LOOP INSERT INTO product (name, description, price) VALUES ('Пицца №' i, 'Описание для пиццы №' i, (random() * 900 + 100)::DECIMAL(10,2)); END LOOP; END; \$\$ LANGUAGE plpgsql

Рисунок 15



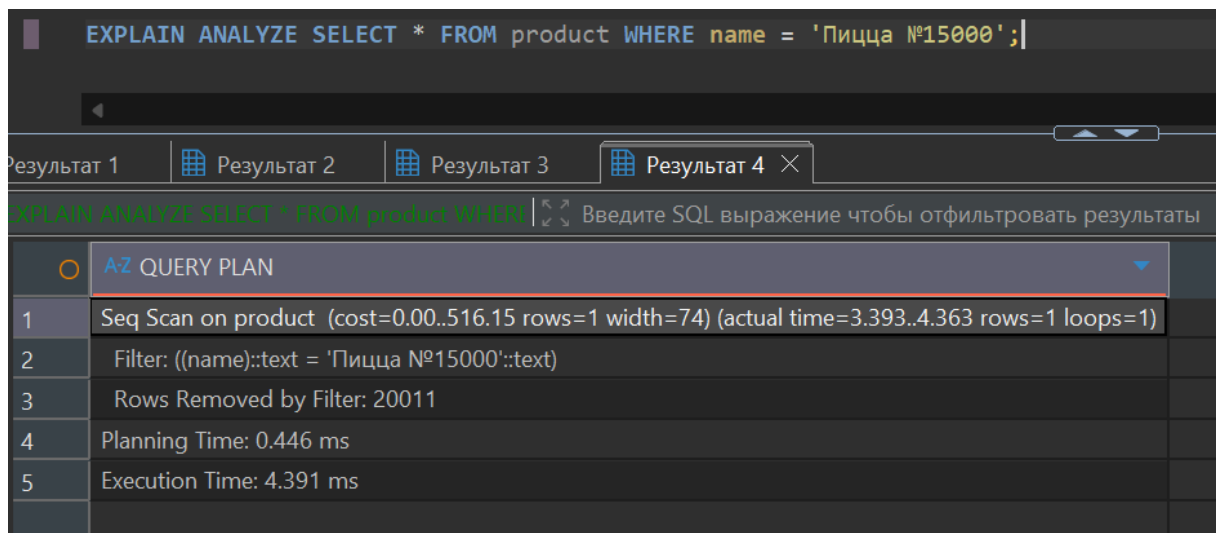
```
SELECT COUNT(*) FROM product;
```

Таблица	123 count
1	20 012

Рисунок 16

1 Анализ и оптимизация

1.1 Стандартный поиск



The screenshot shows the SQL Server Enterprise Manager interface. At the top, a query is entered: `EXPLAIN ANALYZE SELECT * FROM product WHERE name = 'Пицца №15000';`. Below the query editor, there are tabs for 'Результат 1', 'Результат 2', 'Результат 3', and 'Результат 4'. The 'Результат 4' tab is selected, displaying the query plan. The plan is titled 'AZ QUERY PLAN'. It consists of five steps:

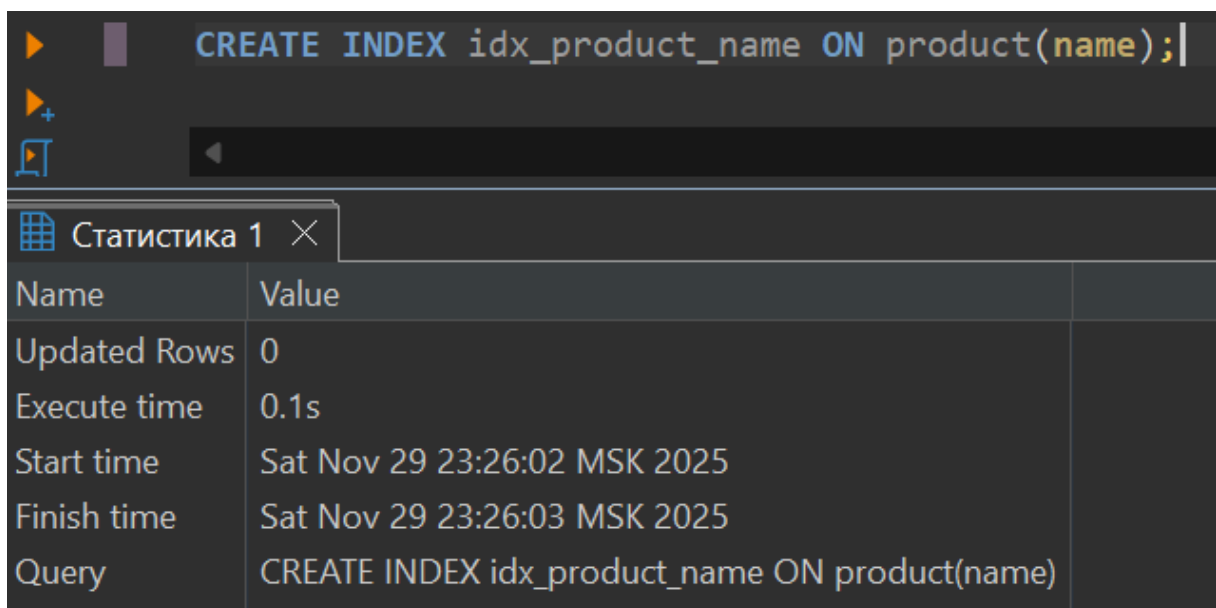
Step	Operation
1	Seq Scan on product (cost=0.00..516.15 rows=1 width=74) (actual time=3.393..4.363 rows=1 loops=1)
2	Filter: ((name)::text = 'Пицца №15000'::text)
3	Rows Removed by Filter: 20011
4	Planning Time: 0.446 ms
5	Execution Time: 4.391 ms

Рисунок 17 – Медленный поиск

Тип плана – Seq Scan.

Время выполнения – Execution Time: 4.391 ms.

Это время будет линейно расти с увеличением таблицы.



The screenshot shows the SQL Server Enterprise Manager interface. At the top, a query is entered: `CREATE INDEX idx_product_name ON product(name);`. Below the query editor, there is a tab labeled 'Статистика 1'. The tab is selected, displaying a table with the following data:

Name	Value
Updated Rows	0
Execute time	0.1s
Start time	Sat Nov 29 23:26:02 MSK 2025
Finish time	Sat Nov 29 23:26:03 MSK 2025
Query	CREATE INDEX idx_product_name ON product(name)

Рисунок 18 – Создание индекса для первого сценария

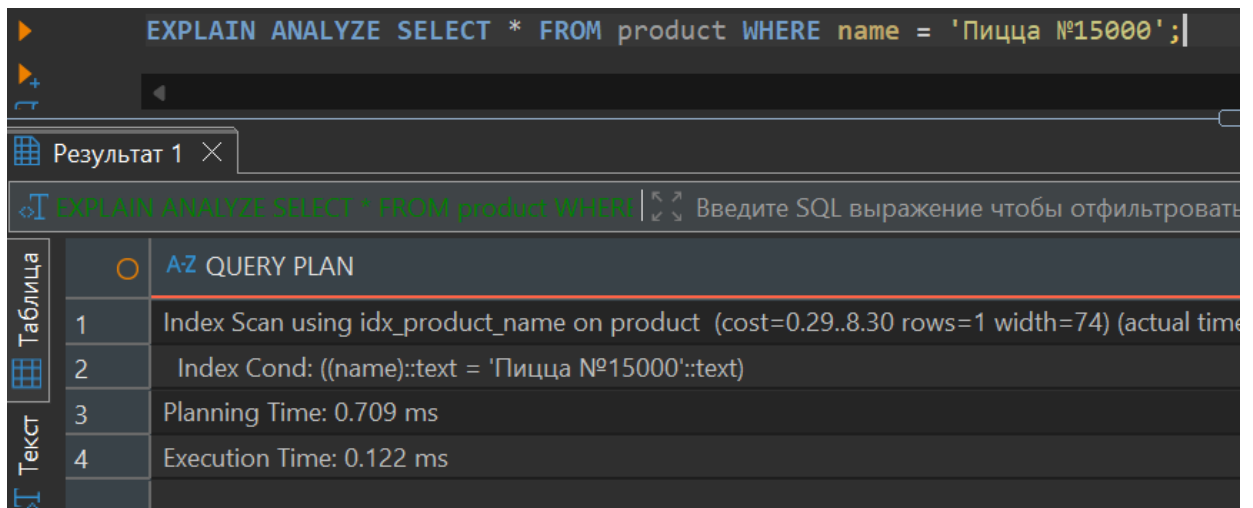


Рисунок 19 – Оптимизированный поиск

Тип плана – Index Scan.

Время выполнения – Execution Time: 0.122 ms.

Это время будет незначительно расти с увеличением таблицы.

Таблица 1 – Сравнительная таблица для первого сценария

Метрика	До оптимизации	После оптимизации	Вывод
План (Оператор)	Seq Scan	Index Scan	Выбор отличается
Execution Time	4.391 ms	0.122 ms	Запрос ускорился примерно в 36 раз

1.2 Поиск в любом регистре

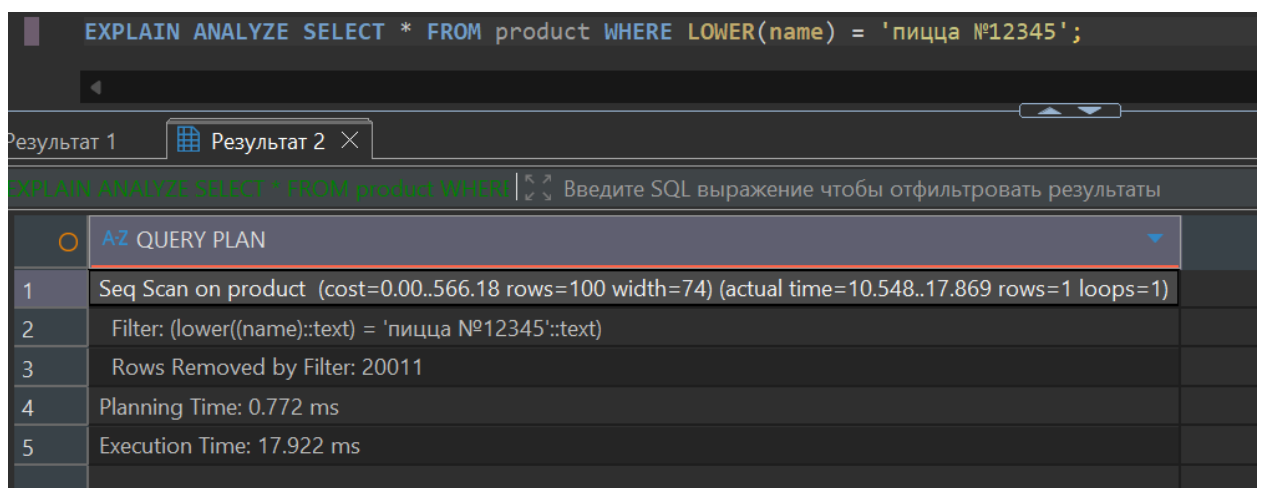


Рисунок 20 – Медленный поиск в любом регистре

Тип плана – Seq Scan.

Время выполнения – Execution Time: 17.922 ms.

Это время будет линейно расти с увеличением таблицы.

CREATE INDEX idx_product_name_lower ON product (LOWER(name));		
Статистика 1 X		
Name	Value	
Updated Rows	0	
Execute time	0.1s	
Start time	Sun Nov 30 00:31:19 MSK 2025	
Finish time	Sun Nov 30 00:31:19 MSK 2025	
Query	CREATE INDEX idx_product_name_lower ON product (LOWER(name))	

Рисунок 21 – Создание индекса для второго сценария

EXPLAIN ANALYZE SELECT * FROM product WHERE LOWER(name) = 'пицца №12345';		
Результат 1 X		
EXPLAIN ANALYZE SELECT * FROM product WHERE Введите SQL выражение чтобы отфильтровать результаты		
AZ QUERY PLAN		
1	Bitmap Heap Scan on product (cost=5.06..202.41 rows=100 width=74) (actual time=0.099..0.100 rows=1 loops=1)	
2	Recheck Cond: (lower((name)::text) = 'пицца №12345'::text)	
3	Heap Blocks: exact=1	
4	-> Bitmap Index Scan on idx_product_name_lower (cost=0.00..5.04 rows=100 width=0) (actual time=0.074..0.074 rows=1 loops=1)	
5	Index Cond: (lower((name)::text) = 'пицца №12345'::text)	
6	Planning Time: 0.459 ms	
7	Execution Time: 0.128 ms	

Рисунок 22 – Оптимизированный поиск в любом регистре

Тип плана – Bitmap Heap Scan.

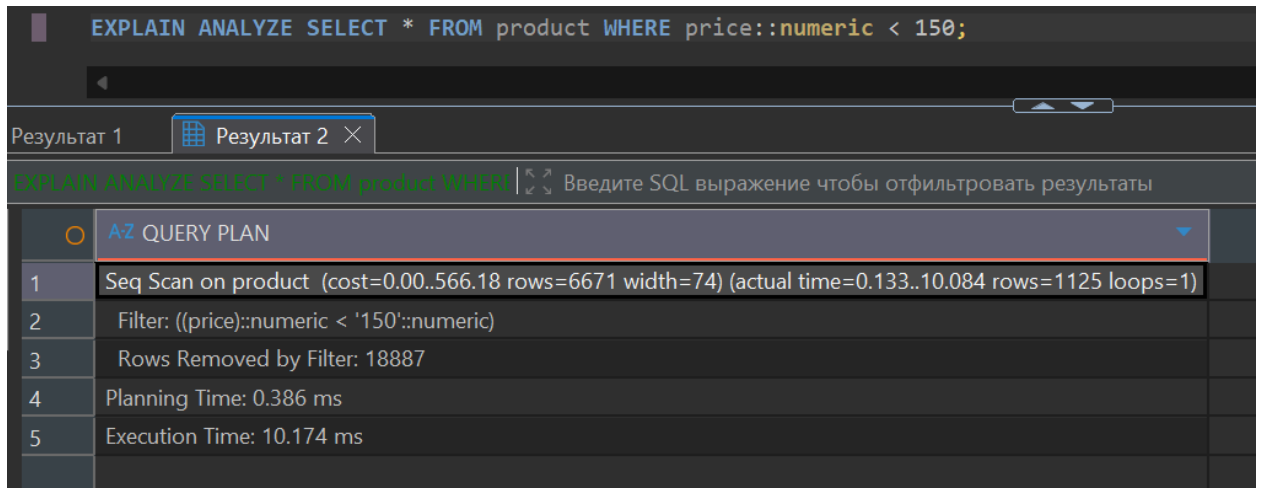
Время выполнения – Execution Time: 0.128 ms.

Это время будет незначительно расти с увеличением таблицы.

Таблица 2 – Сравнительная таблица для второго сценария

Метрика	До оптимизации	После оптимизации	Вывод
План (Оператор)	Seq Scan	Bitmap Heap Scan	Выбор отличается
Execution Time	17.922 ms	0.128 ms	Запрос ускорился примерно в 140 раз

1.3 Фильтр по цене



The screenshot shows a database interface with a query: `EXPLAIN ANALYZE SELECT * FROM product WHERE price::numeric < 150;`. Below the query, there are tabs for 'Результат 1' and 'Результат 2'. The 'Результат 2' tab is active, displaying the query plan. The plan consists of five steps:

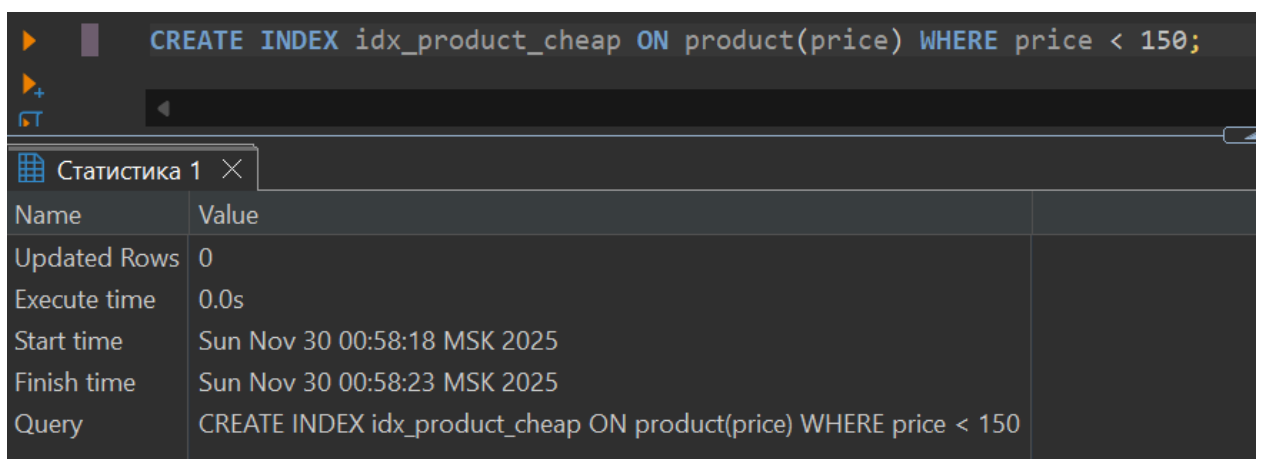
Step	Operation
1	Seq Scan on product (cost=0.00..566.18 rows=6671 width=74) (actual time=0.133..10.084 rows=1125 loops=1)
2	Filter: ((price)::numeric < '150'::numeric)
3	Rows Removed by Filter: 18887
4	Planning Time: 0.386 ms
5	Execution Time: 10.174 ms

Рисунок 23 – Медленный фильтр по цене

Тип плана – Seq Scan.

Время выполнения – Execution Time: 10.174 ms.

Это время будет линейно расти с увеличением таблицы.



The screenshot shows a database interface with a query: `CREATE INDEX idx_product_cheap ON product(price) WHERE price < 150;`. Below the query, there are tabs for 'Статистика 1' and 'Статистика 2'. The 'Статистика 1' tab is active, displaying the statistics for the query execution.

Name	Value
Updated Rows	0
Execute time	0.0s
Start time	Sun Nov 30 00:58:18 MSK 2025
Finish time	Sun Nov 30 00:58:23 MSK 2025
Query	CREATE INDEX idx_product_cheap ON product(price) WHERE price < 150

Рисунок 24 – Создание индекса для третьего сценария

Таблица	Текст
	A-Z QUERY PLAN
1	Bitmap Heap Scan on product (cost=31.57..398.96 rows=6671 width=82) (actual time=0.195..0.836 rows=1125 loops=1)
2	Recheck Cond: (price < '150'::numeric)
3	Heap Blocks: exact=264
4	-> Bitmap Index Scan on idx_product_cheap (cost=0.00..29.90 rows=6671 width=0) (actual time=0.157..0.158 rows=112)
5	Planning Time: 0.628 ms
6	Execution Time: 0.908 ms

Рисунок 25 – Оптимизированный фильтр по цене

Тип плана – Bitmap Heap Scan.

Время выполнения – Execution Time: 0.908 ms.

Это время будет незначительно расти с увеличением таблицы.

Таблица 3 – Сравнительная таблица для второго сценария

Метрика	До оптимизации	После оптимизации	Вывод
План (Оператор)	Seq Scan	Bitmap Heap Scan	Выбор отличается
Execution Time	10.174 ms	0.908 ms	Запрос ускорился примерно в 11 раз

2 Демонстрация атомарной транзакции (COMMIT)

```

select
  p_i.id_ingredient as ingredient_id,
  p_i.ingredient_weight as required,
  s_i.ingredient_weight as remaining
FROM storage_ingredient as s_i
join product_ingredient as p_i on s_i.id_ingredient = p_i.id_ingredient
WHERE s_i.id_storage = 1 AND p_i.id_product = 1;

```

	ingredient_id	required	remaining
1	2	200	4 400
2	1	500	8 500
3	3	150	2 550

Рисунок 26 – Проверка исходных данных, часть 1

<pre> SELECT * FROM order_product WHERE id_order = 2; </pre>				
order_product 1 X				
SELECT * FROM order_product WHERE id_order = 2 Введите SQL выражение чтобы отфильтровать результаты				
	123 id_order_product	123 id_order	123 id_product	123 product_count
1	3	2	2	1

Рисунок 27 – Проверка исходных данных, часть 2

<pre> BEGIN; UPDATE storage_ingredient as s_i SET ingredient_weight = s_i.ingredient_weight - p_i.ingredient_weight FROM product_ingredient as p_i WHERE s_i.id_ingredient = p_i.id_ingredient AND p_i.id_product = 1 AND s_i.id_storage = 1; INSERT INTO order_product (id_order, id_product, product_count) VALUES (2, 1, 1); COMMIT; </pre>	
Статистика 1 X	
Name	Value
Queries	4
Updated Rows	4
Execute time	0.1s
Fetch time	0.0s
Total time	0.1s
Start time	2025-11-30 04:42:17.883
Finish time	2025-11-30 04:42:18.042

Рисунок 28 – Транзакция добавления в существующий заказ блюда

	123 ingredient_id	123 required	123 remaining
1	2	200	4 200
2	1	500	8 000
3	3	150	2 400

Рисунок 29 – Проверка данных после выполнения транзакции, часть 1

	123 id_order_product ▼	123 id_order ▼	123 id_product ▼	123 product_count ▼
1	3	2	2	1
2	30	2	1	1

Рисунок 30 – Проверка данных после выполнения транзакции, часть 2

3 Демонстрация отката транзакции (ROLLBACK)

```

BEGIN;
UPDATE storage_ingredient as s_i
SET
    ingredient_weight = s_i.ingredient_weight - p_i.ingredient_weight
FROM
    product_ingredient as p_i
WHERE
    s_i.id_ingredient = p_i.id_ingredient
    AND p_i.id_product = 1
    AND s_i.id_storage = 1;

INSERT INTO order_product (id_order_product, id_order, id_product, product_count)
VALUES (1, 2, 1, 1);
COMMIT;

```

Execution Error

Error occurred during SQL script execution

<< Сведения Остановить Повторить Пропустить Skip all

SQL Error [23505]: ERROR: duplicate key value violates unique constraint "order_product_pkey"
 Подробности: Key (id_order_product)=(1) already exists.
 ERROR: duplicate key value violates unique constraint "order_product_pkey"
 Подробности: Key (id_order_product)=(1) already exists.

Рисунок 31 – Вызов ошибки при выполнении транзакции

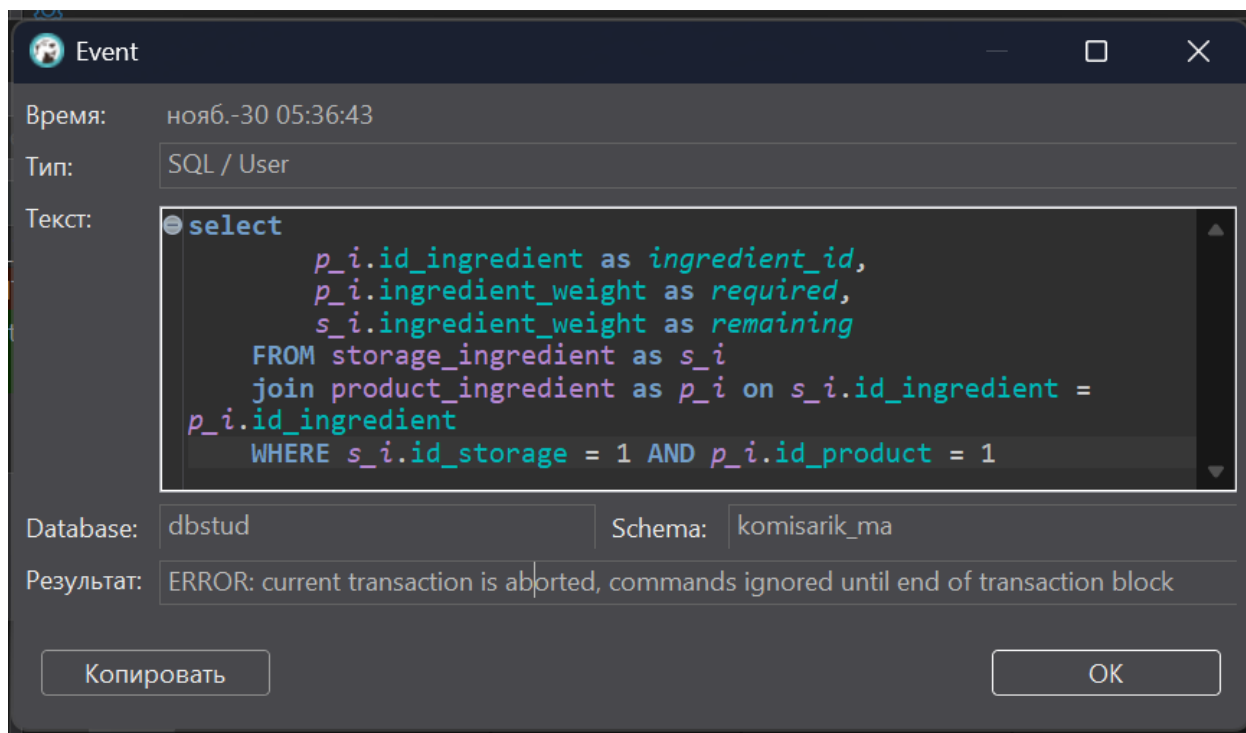


Рисунок 32 – Ошибка при выполнении запроса в ошибочной транзакции

Выполним команду ROLLBACK.

	123 ingredient_id	123 required	123 remaining
1	2	200	4 200
2	1	500	8 000
3	3	150	2 400

Рисунок 33 – Проверка данных после ROLLBACK, часть 1

	123 id_order_product	123 id_order	123 id_product	123 product_count
1	3	2	2	1
2	30	2	1	1

Рисунок 34 – Проверка данных после ROLLBACK, часть 2

4 Моделирование аномалии «Неповторяемое чтение»

```
BEGIN;  
    SELECT price FROM product WHERE name = 'Pepperoni Pizza';  
  
    SELECT pg_sleep(10);  
  
    SELECT price FROM product WHERE name = 'Pepperoni Pizza';  
COMMIT;
```

Рисунок 35 – Сценарий долгой транзакции

```
UPDATE product SET price = 15000 WHERE name = 'Pepperoni Pizza';
```

Рисунок 36 – Запрос на изменение цены

The screenshot shows a database client interface with a tab labeled 'product 1'. Below the tab, there is a search bar with the text 'Введите SQL выражение чтобы отфильтровать результаты'. Below the search bar, there is a table with the following data:

Таблица	price
1	12.99

Рисунок 37 – Цена в начале запроса

The screenshot shows a database client interface with a tab labeled 'product 1 (3)'. Below the tab, there is a search bar with the text 'Введите SQL выражение чтобы отфильтровать результаты'. Below the search bar, there is a table with the following data:

Таблица	price
1	15000.00

Рисунок 38 – Цена в конце запроса

5 Устранение аномалии «Неповторяемое чтение»

```
BEGIN;  
    SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;  
  
    SELECT price FROM product WHERE name = 'Pepperoni Pizza';  
  
    SELECT pg_sleep(10);  
  
    SELECT price FROM product WHERE name = 'Pepperoni Pizza';  
COMMIT;
```

Рисунок 39 – Сценарий долгой транзакции с повторяемым чтением

product 1		Результат 1 (2)	product 1 (3)	Статистика 1
COMMIT Введите SQL выражение чтобы отфильтровать результаты				
Таблица		123 price		
	1	12.99		

Рисунок 40 – Цена в начале запроса

product 1		Результат 1 (2)	product 1 (3)	Статистика 1
COMMIT Введите SQL выражение чтобы отфильтровать результаты				
Таблица		123 price		
	1	12.99		

Рисунок 41 – Цена в конце запроса